

A Formal Definition of Instructions in a Domain-Specific Language

Prepared with Claude (Anthropic)

June 20, 2026

Abstract

An *instruction* (or *command*) sits at the intersection of two things: a symbol in the grammar of a domain-specific language, and a behavior in the program that language describes. This note defines it self-containedly – its abstract syntax as a named operator carrying optional parameters, the forms those parameters take, its reading across the standard grammar metasyntaxes, and its semantics as a function from arguments to a computation – with no dependence on any external apparatus.

Instructions

An **instruction** (equivalently, a **command**) is a node of the abstract syntax: a named operator that may carry data. Its name is the node’s head; its data are the node’s children. No meaning resides in the name itself – meaning is supplied separately, by an interpretation, developed in the final section.

Given:

- (1) a **name** n , an identifier drawn from a namespace $\mathcal{N}$;
- (2) a **parameter** a : a slot the instruction may carry, equipped with a **domain** D_a (its admissible values) and a marking *required* or *optional*; an optional parameter additionally admits the **absent** value $\perp \notin D_a$ and may name a **default** $d_a \in D_a$;
- (3) a **parameter profile** $\text{prof}(n)$: the finite, possibly empty, set of parameters declared for the name n ,

one defines:

- (4) an **instruction** c : a name $\text{name}(c) \in \mathcal{N}$ together with a **binding** β_c assigning to each $a \in \text{prof}(c)$ a value $\beta_c(a) \in D_a \cup \{\perp\}$, with $\perp$ permitted only where a is optional. The **arity** of c is $|\text{prof}(c)|$, writing $\text{prof}(c) := \text{prof}(\text{name}(c))$.

The binding is the instruction’s entire variable content: a finite map from its parameters to values, recording which optional parameters are present and what each carries. Flags, attributes, arguments – and command-line “options” in the informal sense – are not separate primitives but *parameters* differing only in the shape of their domain; an instruction “may or may not” carry them exactly as its profile is non-empty or empty. An instruction of empty profile is a **bare command**, an atom

with a name and nothing more. Being *abstract* syntax, an instruction retains only its name and binding; distinct surface renderings that induce the same binding are the same instruction.

Each parameter falls, by the shape of D_a , into exactly one **mode**:

- (5) **nullary** (*flag*): $D_a = \{\star\}$, a single presence token; the only datum is present-versus-absent.
- (6) **Boolean**: $D_a = \mathbb{B} = \{\text{true}, \text{false}\}$.
- (7) **enumerated**: $D_a = \{u_1, \dots, u_r\}$, a finite enumeration.
- (8) **valued**: D_a is an arbitrary type (the general case).

These nest – nullary is the one-element enumeration, Boolean the two-element one – and are separated only so the semantics can treat their values uniformly.

The forms an instruction takes

The variable content of an instruction is the values its binding records. A few recurring forms are worth naming; each is a statement about the binding alone.

Surface instruction	Domain D_a	Binding contribution (parameter $\mapsto$ value)
flag foo (present)	$\{\star\}$	foo $\mapsto \star$ – present, valueless
do_foo = b	$\mathbb{B}$	do_foo $\mapsto b$, $b \in \{\text{true}, \text{false}\}$
pair do_foo / dont_do_foo	name-selected $\mathbb{B}$	$s \mapsto \varpi(\text{name}(c))$ for a shared setting s
some_command = u	enum $\{u_1, \dots, u_r\}$	some_command $\mapsto u$, $u \in \{u_1, \dots, u_r\}$

Table 1: Each instruction form as a contribution to the binding, graded by D_a .

The **flag** foo carries a single nullary parameter: issued, its binding records presence with no value; unissued, the parameter is absent ($\perp$). Only presence is observable, and presence *is* the content.

The **Boolean** do_foo = b records a truth value directly.

The **polarity pair** is two distinct command names, neither carrying an explicit value, that denote one underlying Boolean setting s with opposite values. A **polarity** ϖ sends each name to a Boolean – $\varpi(\text{do_foo}) = \text{true}$, $\varpi(\text{dont_do_foo}) = \text{false}$ – so the choice of name selects the value of s . The pair do_foo/dont_do_foo and the explicit form do_foo = b therefore carry the *same* content: two concrete syntaxes for one setting. (The two names form an orbit of the fixed-point-free involution that swaps polarity, and ϖ is the $\mathbb{Z}/2$ -valued invariant it induces.)

The **enumerated** some_command = u records a value from a finite domain.

Remark (one scheme). The forms are a single construction graded by $|D_a|$ and by whether the value is retained or suppressed to presence alone: the flag is the one-element domain with its value forgotten, the Boolean the two-element domain, the enumeration the finite domain, the valued parameter the unrestricted one. A multi-parameter instruction is just their product over the profile.

Reading the definition against a grammar

The construction does not depend on how a DSL’s surface is written down. Fix a context-free grammar $G = (N, \Sigma, P, S)$ – nonterminals N , terminals Σ , productions P , start symbol S – presented in any of the usual notations. The instruction layer is the *abstract* content of G ; the notation is only its concrete surface. The dictionary is:

- (1) a **name** is a terminal keyword in Σ (equivalently, the production the keyword selects);
- (2) an **instruction** is a nonterminal $\langle cmd \rangle$ together with its production; the **profile** is that production’s right-hand side;
- (3) a **parameter** a is a nonterminal A_a , and its **domain** is the language that nonterminal generates, $D_a = \mathcal{L}(A_a)$;
- (4) a **binding** β_c is the attribution of a parse tree: a filled parameter slot carries the value parsed there; an optional slot taken empty carries $\perp$;
- (5) the **mode** is read off A_a ’s production – a singleton right-hand side (nullary), `true` | `false` (Boolean), a finite alternation (enumerated), or an arbitrary sublanguage (valued).

Taking $D_a = \mathcal{L}(A_a)$ is what frees the definition from any particular DSL: each language fixes its own parameter languages, and the rest of the development is identical over all of them.

The notations differ only in surface. Each metasyntax expresses the same operators, so one production translates uniformly:

	BNF	EBNF	ABNF	PEG
definition	<code>::=</code>	<code>=</code>	<code>=</code>	<code>←</code>
alternation	<code> </code>	<code> </code>	<code>/</code>	<code>/ (ordered)</code>
optional (X)	<code>X ε</code>	<code>[X]</code>	<code>[X]</code>	<code>X?</code>
repetition (0+)	recursion	<code>{X}</code>	<code>*X</code>	<code>X*</code>
terminal	literal	<code>"lit"</code>	<code>%xHH, "lit"</code>	<code>"lit", [class]</code>
nonterminal	<code><name></code>	<code>name</code>	<code>rulename</code>	<code>Name</code>

Table 2: One production across the four metasyntaxes. Alternation is *unordered* in BNF, EBNF, and ABNF – a symmetric choice whose ambiguity is resolved outside the grammar – but *ordered* in PEG, where `/` is a committed, first-match choice.

Presence as a single bit. An instruction “whose presence or absence is enough” is exactly the optionality operator: $X | \varepsilon$ in BNF, $[X]$ in EBNF and ABNF, $X?$ in PEG. Its parameter is nullary, or a parameter whose value is discarded, so its binding records only presence; the syntactic domain may be a singleton or richer, but the observable content is the single bit present-versus-absent.

Precedence. Priority enters at two independent layers.

Syntactic. When several alternatives could apply, a choice must be made. PEG fixes it inside the grammar: e_1/e_2 tries the left alternative first and commits on success, so precedence is the order of alternatives and the grammar is unambiguous by construction. BNF, EBNF, and ABNF leave alternation symmetric (ABNF’s `/` is set union, not PEG’s ordered choice, despite the shared glyph),

so precedence and associativity are imposed from outside – by stratifying the grammar into levels ($\langle expr \rangle$, $\langle term \rangle$, $\langle factor \rangle$) or by separate precedence declarations.

Semantic. When a program issues conflicting settings for one parameter, a resolution policy decides the winner; the usual policy is sequential override – a later instruction supersedes an earlier one. The two layers are dual instances of one device, a total order that makes a nondeterministic combine deterministic: PEG’s ordered choice is “leftmost alternative wins” at parse time, sequential override is “latest issuance wins” at execution time.

Attributes. The denotation is an **attribute grammar** over G . Each parameter nonterminal A_a synthesizes its value $\beta_c(a) \in D_a \cup \{\perp\}$; the production for $\langle cmd \rangle$ carries a semantic rule assembling these into the instruction’s binding, and a further rule into its behavior. Defaults d_a and any ambient context – the active namespace, the enclosing scope – travel the other way, as *inherited* attributes. Because every value is synthesized and the only inheritance flows left-to-right, the definition is L -attributed: it is evaluable in a single left-to-right pass.

The behavioral half: instructions as interpreted symbols

The previous sections fixed the *syntactic* half of an instruction – a name, a profile, and the binding it parses to. Its *semantic* half is a function f saying what executing it does. The two are glued along the profile: parsing fills the profile with a binding, and f reads that binding. An instruction is thus one symbol wearing two faces – a grammar symbol and a behavior – paired over the arity they share.

Fix:

- (1) an **effect monad** T and a **result type** R ; a **computation** is an element of $T(R)$. The common command runs only for effect, $R = \mathbf{1}$; specializing T to the State monad $T(R) = St \rightarrow (R \times St)$ makes a computation a state transformer $St \rightarrow St$, while other monads supply exceptions, input/output, or nondeterminism;
- (2) the **binding space**, the dependent sum (a telescope in which each factor may depend on the earlier ones)

$$B = \sum_{a \in A} (D_a(\beta_{<a}) \cup \{\perp\}),$$

degenerating to the flat product $\prod_{a \in A} (D_a \cup \{\perp\})$ when the parameter domains are independent;

- (3) a **validity predicate** $\Phi: B \rightarrow \mathbb{B}$, the conjunction of the cross-parameter constraints – required-present, mutual exclusion, requires/implies – cutting out the **valid bindings**

$$B_{\text{valid}} = \{ \beta \in B \mid \Phi(\beta) \}.$$

Dependencies – one parameter’s *domain* gated by another’s value – live in the Σ -shape of B ; flat constraints – exclusion, co-requirement – live in Φ .

The semantic function. The behavior of an instruction is a map

$$f: B_{\text{valid}} \longrightarrow T(R),$$

total on the valid bindings. Presence or absence of a flag is the choice of $\perp$ in a coordinate, so f is a single function on a (dependent) product – not one function per subset of present flags. The space of candidate behaviors compatible with the symbol’s arity is therefore the exponential

$$F = [B_{\text{valid}} \rightarrow T(R)],$$

and an instruction is its symbol together with a chosen point $f \in F$.

Assembling the behavior parameter-wise. Defining f pointwise would list $|B_{\text{valid}}|$ cases; present it instead from parts linear in the parameters. Give the instruction a **base behavior** f_0 , a function of its required arguments alone, and give each optional parameter a a **modifier** $m_a: D_a \cup \{\perp\} \rightarrow \text{End}(T(R))$, an endo-transformer on computations, with

$$m_a(\perp) = \text{id}.$$

Then

$$f(\beta) = \left(\bigcirc_{a \in A_{\text{opt}}} m_a(\beta(a)) \right) (f_0(\beta|_{\text{req}})),$$

the composite of the present modifiers applied to the base, where A_{opt} is the set of optional parameters and $\beta|_{\text{req}}$ the restriction of β to the required ones. Absence is the identity element, presence applies a modifier, and a combination is their composite: $|A_{\text{opt}}|$ modifiers and one base generate every behavior, the exponential blow-up confined to the *image* $\{f(\beta) : \beta \in B_{\text{valid}}\}$ – the orbit of f_0 under the modifier monoid – which is never written out.

Proposition (order-independence $\iff$ commutativity). The collection of present optional parameters carries no intrinsic order, so the composite above is well defined exactly when the modifiers pairwise commute in $\text{End}(T(R))$; equivalently, the behavior factors through the action of a *commutative* monoid, and parameter order is then semantically irrelevant. When the modifiers do not commute, the composite is made well defined by composing them in the parameter precedence of the preceding section.

Proof sketch. A fold over an unordered set is independent of its enumeration iff its operations commute, which for the monoid $(\text{End}(T(R)), \circ, \text{id})$ is pairwise commutativity of the $m_a(\beta(a))$. Failing that, any total order on the parameters determines the fold, and precedence is the canonical such order. $\square$

Sequencing. A **program** is a finite sequence $c_1; \dots; c_m$ of instructions. Its behavior is the composite of the individual computations in the effect monad,

$$f_{c_m}(\beta_{c_m}) \bullet \dots \bullet f_{c_1}(\beta_{c_1}),$$

where $\bullet$ is composition in T – for the State monad, ordinary composition of state transformers, threading the state through – applied right to left, so c_1 executes first. Sequential override is then a derived property: if two instructions write the same state component, the later computation runs second and its write prevails.

The intersection, precisely. Write Ω for the signature of instruction constructors. The abstract syntax of the first section is the term algebra it freely generates; assigning every constructor a behavior makes computations an Ω -algebra, and the program's denotation is the unique homomorphism from that term algebra into the algebra of computations. The function f is this homomorphism's action at a single constructor. An instruction is therefore one constructor read in both algebras at once – a symbol on the syntactic side, a computation on the semantic side, identified along the profile each must honor. That single arrow, from a symbol to its computation, is the intersection of grammar and behavior the development has been circling.